

El transformer aprende a ver.

Cerramos el RAG. Hoy el mismo transformer entra a las imágenes: parches en vez de palabras (ViT), texto e imagen en un solo espacio (CLIP) y un LLM que mira (LLaVA). Al final: qué ojos le van a poner a SU sistema.

ViT

CLIP

MULTIMODAL

La sesión de hoy.

Recap: resolvemos las dudas del formulario de salida

Del texto a la imagen: ViT, parches como tokens

CLIP: texto e imagen en el mismo espacio • zero-shot  **Lab A**

Modelos multimodales: LLaVA y familia • visual QA  **Lab B**

Patrones de integración visual para SU proyecto • declaración del plan

Hacia S12 (difusión) y la entrega M4 (S13)

Primero, sus dudas.

Cinco temas del formulario: RAGAS y faithfulness, tools vs. RAG vs. agente, DSPy, MCP y el tweet del cierre.

RAGAS en una imagen: cuatro medidores sobre UN caso ya resuelto

no es un flujo ni una técnica — es evaluación: toma (pregunta, contexto, respuesta, referencia) y compara piezas de a dos



Lectura de diagnóstico — cada color señala a un culpable distinto:

- **morado bajo** → falla el RETRIEVAL: vuelvan a S08 (hybrid, reranking, query transformation)
- **rosado bajo** → falla la GENERACIÓN: prompt estricto, válvula de escape, menos ruido en el top-k

Lo que las cuatro tienen en común

- Cada métrica es UN juez que compara DOS piezas del caso.
- Todas van de 0 a 1. Se calculan DESPUÉS de que el sistema respondió.
- Las cuatro juntas dicen dónde se rompió: antes o después del prompt.
- No cambian el sistema: lo radiografían. (Los labs A y B construyen; el lab C mide lo que A y B construyeron.)

"¿Faithfulness solo sirve en RAGAS?" → No.

Es un CONCEPTO (grounding): ¿la respuesta se sostiene en algo que le dimos? Solo necesita dos cosas: una RESPUESTA y un CONTEXTO. Funciona en cualquier arquitectura que tenga ambas:

RAG: contexto = chunks

Agente con tools: contexto = observaciones de las tools

Resumen: contexto = el documento

Chat con archivo: contexto = el archivo

RAGAS la empaquetó; ustedes la pueden calcular donde quieran (como en el lab C).

Faithfulness por dentro (lo que hicieron en el lab C):

- 1 · partir la RESPUESTA en afirmaciones
- 2 · por cada una, preguntarle al juez: "¿el CONTEXTO la respalda? sí/no"
- 3 · faithfulness = respaldadas / total

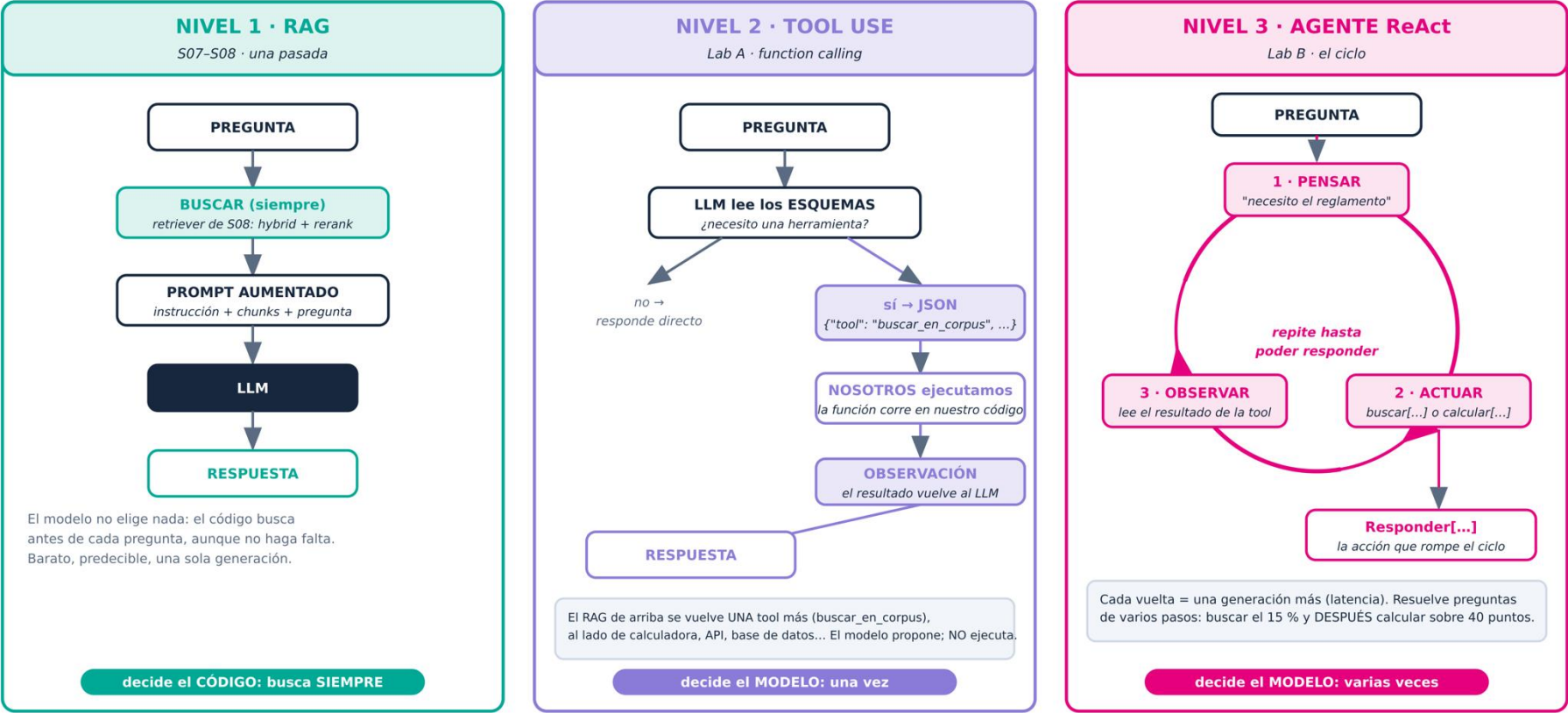
Ejemplo: 3 afirmaciones, 2 respaldadas → 0.67. La tercera "sonaba bien" pero salió de la memoria del modelo, no del contexto: con otro contexto, sería una alucinación. Por eso faithfulness es la que más las atrapa.

El harness dice SI acertaron. RAGAS dice POR QUÉ: retrieval (morado) o generación (rosado).

se reportan juntos en M3 — uno sin el otro cuenta media historia

RAG, tools y agente: el mismo modelo, tres niveles de autonomía

lo que cambia no es la búsqueda ni el LLM — es QUIÉN decide actuar y CUÁNTAS veces



	RAG	TOOL USE	AGENTE ReAct
¿Quién decide buscar?	el código, siempre	el modelo, una vez	el modelo, en cada vuelta
Generaciones por pregunta	1	1-2	una por paso (3, 4, 5...)
Latencia y costo	bajo	medio	alto
Cuándo conviene	la pregunta se responde con los documentos	hace falta un cálculo, un dato vivo o elegir fuente	varios pasos encadenados
Cómo se justifica	es el baseline	si el harness sube	solo si el harness sube DE VERDAD

Mismo LLM, mismo retriever. Sube de nivel solo cuando el harness lo confirme — y RAGAS se aplica DESPUÉS, a cualquiera de los tres.

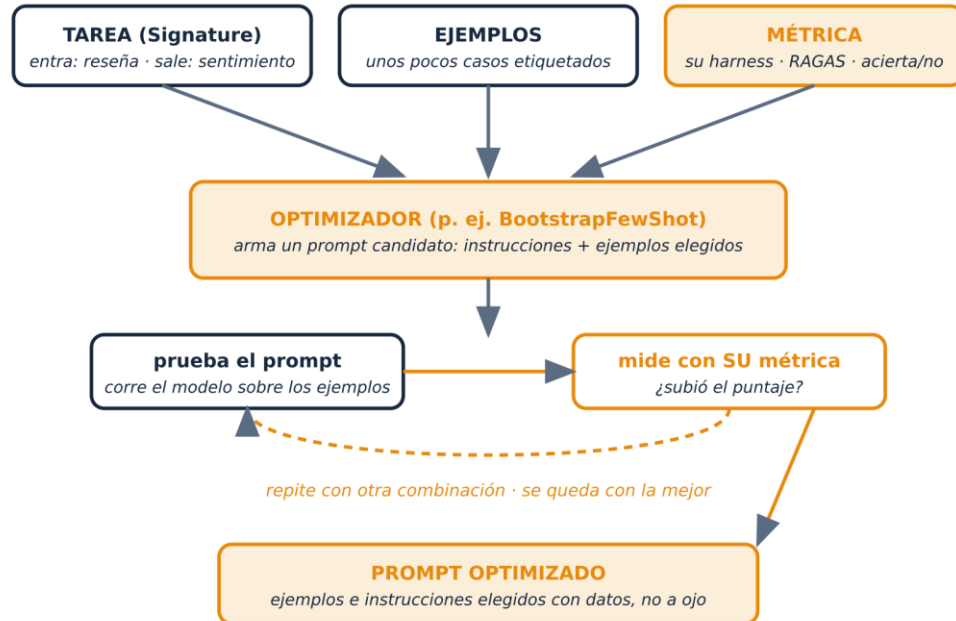
RAGAS no construye: mide. Los labs A y B son el sistema; el lab C es el termómetro.

Dos cosas que vienen después: DSPy y MCP

ninguna reemplaza lo que ya saben — las dos se apoyan en el harness y en el esquema de tools del lab A

DSPy · optimizar el prompt, no escribirlo

lo que hicieron a mano en S07-S10, hecho por búsqueda guiada por su métrica



Analogía: es grid search, pero el "hiperparámetro" es el prompt.

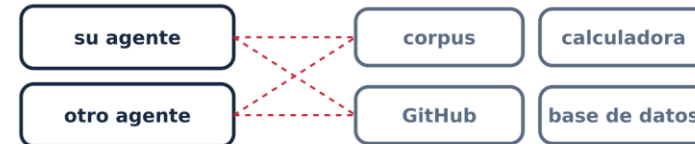
La regla: sin una métrica confiable no hay nada que optimizar. Por eso va DESPUÉS de M2 (harness) y M3 (RAGAS): DSPy es un consumidor de su báscula.

Notebook extra S10_Extra_DSPy: corre local con Ollama, sin API keys.

MCP · Model Context Protocol: el USB-C de las tools

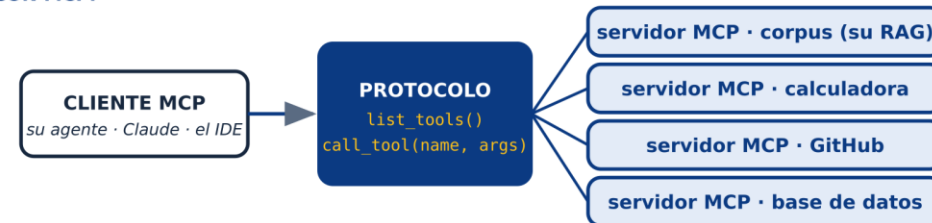
un estándar abierto para que cualquier modelo use cualquier herramienta, sin código a la medida

ANTES (lo del lab A, multiplicado):



cada agente escribe su propio esquema y su propio código para cada tool: n agentes $\times$ m tools cables a la medida

CON MCP:



Cada tool se expone UNA vez como servidor MCP; cualquier cliente la descubre y la llama igual. El esquema que escribieron en el lab A (nombre, descripción, parámetros) es exactamente lo que MCP estandariza. El bucle propone → ejecuta → observa no cambia: cambia cómo se ENCHUFAN.

Para su proyecto: no es obligatorio. Pero si en S15 despliegan su RAG como servidor MCP, cualquier asistente (Claude, Cursor, un agente propio) lo usa sin adaptadores.

DSPy optimiza el PROMPT con su métrica · MCP estandariza cómo se ENCHUFAN las tools.

los dos dan por sentado lo que ustedes ya construyeron: una báscula (M2) y tools con esquema (lab A)

 **Sam Altman**   @sama

today we launched ChatGPT. try talking with it here:

chat.openai.com

2:38 PM · Nov 30, 2022

4.2K 23K 96K 38K

Relevant ▾ View quotes >

 Post your reply Reply

 **Sam Altman**   @sama · Nov 30, 2022

language interfaces are going to be a big deal, i think. talk to the computer (voice or text) and get what you want, for increasingly complex definitions of "want"!

this is an early demo of what's possible (still a lot of limitations--it's very much a research release).

37 186 4K

 **Sam Altman**   @sama · Nov 30, 2022

soon you will be able to have helpful assistants that talk to you, answer questions, and give advice. later you can have something that goes off and does tasks for you. eventually you can have something that goes off and discovers new knowledge for you.

114 640 6.3K

 **Sam Altman**   @sama · Nov 30, 2022

but this same interface works for all of that. this is something that scifi really got right; until we get neural interfaces, language interfaces are probably the next best thing.

41 130 2.3K

 **Sam Altman**   @sama · Nov 30, 2022

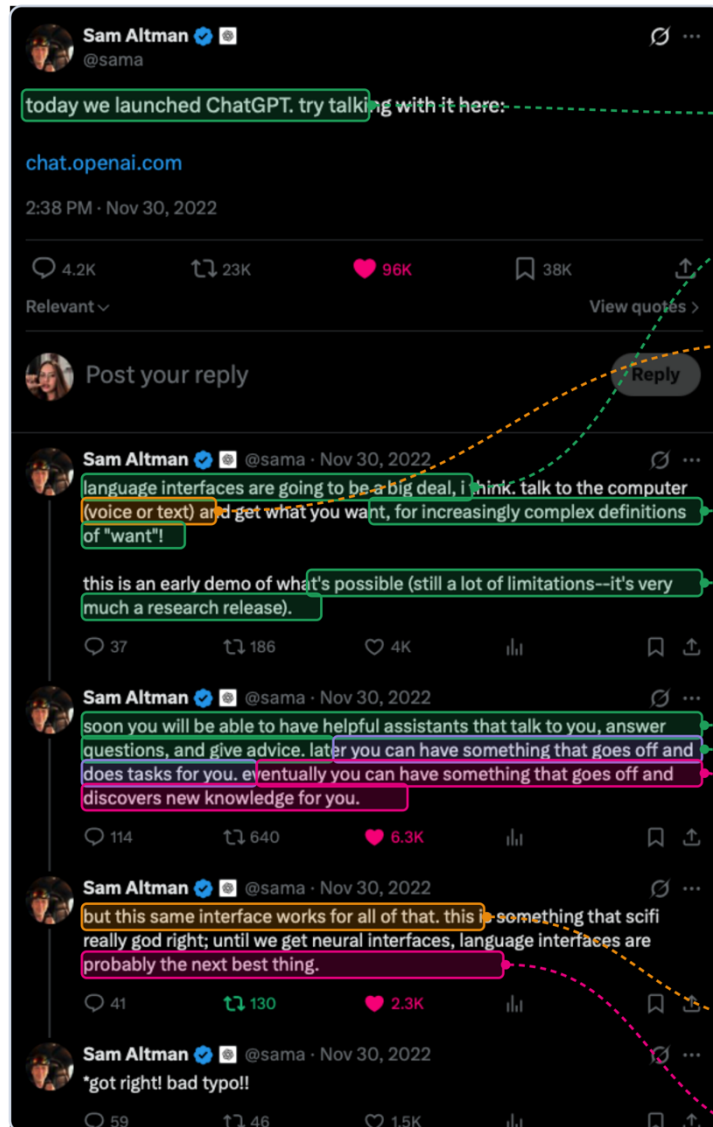
*got right! bad typo!!

59 46 1.5K

<https://x.com/sama/status/1598038815599661056>

El tweet del cierre, leído con lo que ya saben

un hilo de nov. 2022 · cada frase es una parada del curso: unas ya las pasaron, otras vienen



YA LO APRENDIERON **HOY · S11** **LO QUE VIENE** (las marcas van sobre la frase exacta)

S03 · S04 Familias de modelos + fine-tuning
ChatGPT = un decoder-only afinado con instrucciones (RLHF). Ustedes hicieron la versión pequeña: Qwen-Instruct + LoRA sobre su dominio (entrega M1).

S02 · S03 El transformer que genera
"Talk to the computer" = tokens → self-attention → un token a la vez.
La interfaz de lenguaje ES el decoder que abrieron en la caja.

S11 · HOY Voz o texto... o imagen
"(voice or text)": hoy el mismo modelo lee imágenes (VIT, CLIP, un LLM que mira).
Voz es la misma receta con otro encoder: tokens que no vienen de palabras.

S05 · S06 "Definiciones de want cada vez más complejas"
= ¿qué es una BUENA respuesta? Esa pregunta es el Módulo 2 entero:
métrica clásica + LLM-juez + criterio de dominio → su harness y su eval set.

S07 · S10 Las limitaciones tienen nombre
Alucinación, cutoff, contexto ignorado. Se atacan con RAG (buscar antes de responder) y se MIDEN: faithfulness, precision/recall del contexto (RAGAS).

S07 · S08 Asistentes que responden y aconsejan: RAG
Con SUS documentos, no con la memoria del modelo: chunking, embeddings, hybrid search, reranking, query transformation — y el delta en el harness.

S10 "Goes off and does tasks": tools y ReAct
Function calling: el modelo propone la llamada, nosotros ejecutamos.
ReAct: pensar → actuar → observar hasta responder. Su RAG es una tool más.

FRONTERA "Discovers new knowledge": agentes autónomos
Multi-step, memoria, self-RAG (profundización S10). Fuera del curso, pero ya tienen el ciclo y la báscula para saber si un agente de verdad aporta.

S11 · S12 "The same interface works for all of that"
Texto e imagen en el mismo espacio (CLIP) y tokens visuales dentro del LLM: una sola interfaz para ver, leer y — en S12 — dibujar.

S14 · S16 "The next best thing" solo si se usa
Una interfaz vale cuando alguien la usa: destilar, cuantizar, desplegar, observar con W&B y demostrarlo en el demo day (5 NOV).

Ann: <https://www.ucsf.edu/news/2023/08/425986/how-artificial-intelligence-gave-paralyzed-woman-her-voice-back>

Neuralink: <https://neuralink.com/updates/two-years-of-telepathy/>

Brain2Qwerty: <https://facebookresearch.github.io/brain2qwerty/>

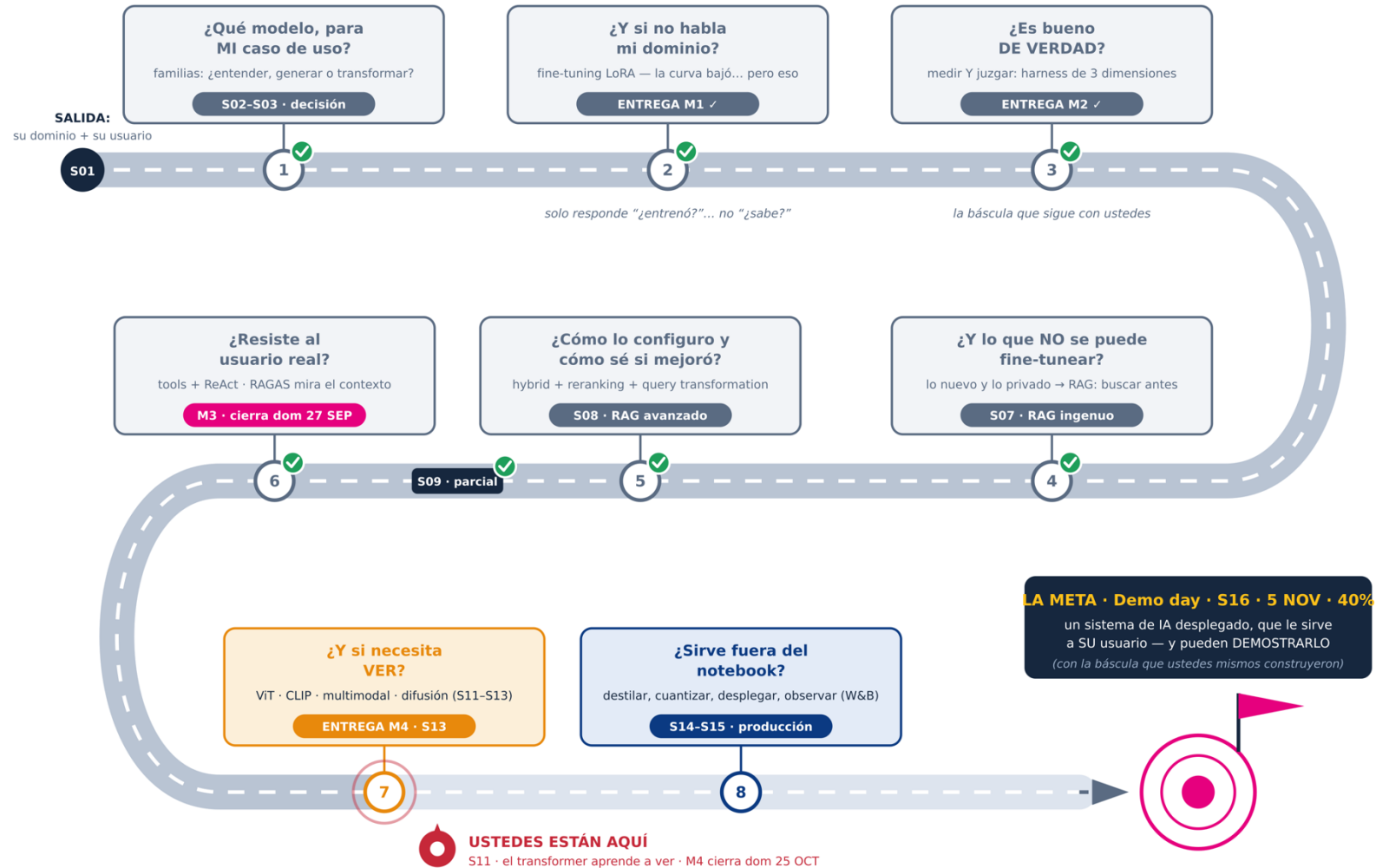
Panorama actual: <https://pdpspectra.com/blog/brain-computer-interfaces-neuralink-2026/>

¿Dónde estamos?

El mapa, actualizado: M3 cierra este domingo y hoy arranca la parada 7.

El mapa del semestre: el RAG quedó atrás, el sistema aprende a ver

lo recorrido en gris · lo que falta en color · la meta no cambió



Lo que falta: M3 cierra dom 27 SEP · M4 dom 25 OCT (10%) · despliegue S15 · demo day 5 NOV (40%) = M1+M2+M3+M4 + producción

Una imagen son 16×16 palabras.

El encoder de S03 no cambia. Lo que cambia es de dónde salen los tokens.

Ya lo usaron tres veces. Hoy, la cuarta.

ENTENDER · S03

encoder-only (BERT): leer un texto y sacar una representación. Lo usaron como embeddings en S05–S07.

GENERAR · S03–S04

decoder-only (Qwen, Llama): escribir token a token. Su generador desde M1; lo fine-tunearon con LoRA.

JUZGAR Y RERANKEAR · S06 · S08

el LLM como juez del harness; el cross-encoder que lee (pregunta, chunk) juntos y puntúa.

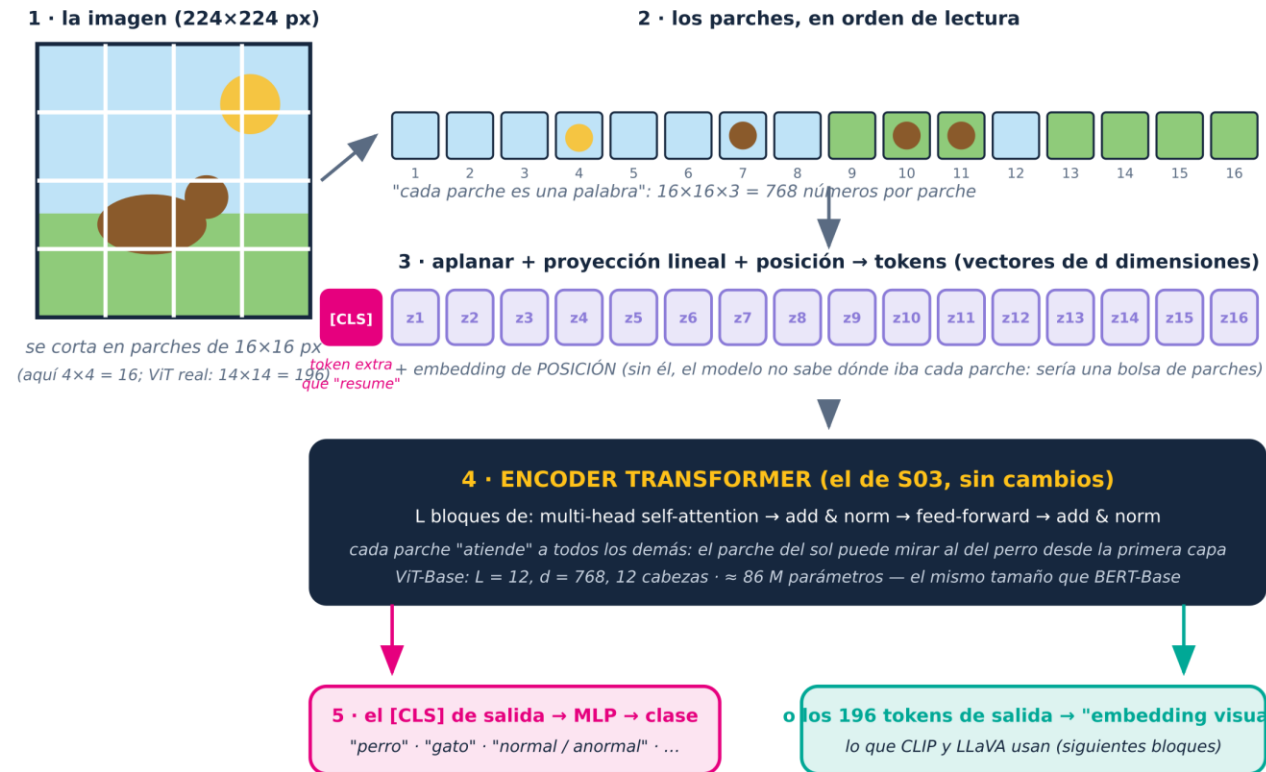
VER · S11

el mismo encoder, pero los tokens son parches de una imagen. Y con CLIP, texto e imagen comparten espacio.

La pregunta de hoy no es "¿cómo funciona la visión?" sino "¿qué tiene que cambiar para que lo que ya saben leer una imagen?". Respuesta corta: casi nada.

ViT: una imagen son 16×16 palabras

el mismo encoder de S03 — lo único nuevo es cómo se fabrican los tokens



Texto vs. imagen: qué cambia

pieza	BERT (texto)	VIT (imagen)
tokenizador	BPE sobre caracteres	cortar en parches 16x16
embedding	tabla de vocabulario	proyección lineal del parche
posición	1-D (orden)	1-D aprendida (o 2-D)
encoder	igual	igual
salida	[CLS] o cada token	[CLS] o cada parche

ViT vs. CNN: el trade-off

CNN: trae "de fábrica" que lo cercano importa (convoluciones locales). Aprende con pocos datos.
ViT: no asume nada; mira toda la imagen desde la capa 1. Con pocos datos, pierde; con muchos (o preentrenado), gana y escala mejor.
Regla para su proyecto: usen ViT PREENTRENADO.

Cuarto oficio del mismo transformer:

entender · generar · juzgar/rerankear · VER

Nada nuevo por dentro: parches en vez de palabras, y el resto es el encoder que ya conocen.

Dosovitskiy et al., 2021 — "An Image is Worth 16×16 Words"

¿Cuándo ViT y cuándo una CNN?

CNN (LO QUE VIERON EN REDES NEURONALES)

Sesgo inductivo de fábrica

Las convoluciones asumen que lo cercano importa y que un patrón vale igual en cualquier posición. Con pocos datos (miles de imágenes) eso ayuda: aprende rápido y generaliza.

- ✓ Pocos datos, poco cómputo, tareas locales (bordes, texturas).
- ✗ Le cuesta relacionar partes lejanas de la imagen.

VIT

No asume nada; lo aprende todo

Cada parche atiende a todos desde la capa 1. Sin sesgo previo, con pocos datos pierde contra la CNN; con muchos (o preentrenado en millones), la supera y escala mejor.

- ✓ Preentrenado, transfiere a casi cualquier dominio; se conecta con texto (CLIP).
- ✗ Entrenarlo desde cero está fuera de su alcance (y del de casi todos).

Para su proyecto la decisión ya está tomada: ViT PREENTRENADO. Y si necesitan adaptarlo, la receta es la de S04: LoRA sobre las capas de atención del ViT, con su dataset de imágenes del dominio.

Referencia: ViT-Base = 86 M parámetros · el mismo tamaño que BERT-Base. No es casualidad: es el mismo encoder.

Texto e imagen en el mismo espacio.

Dos encoders, un contrato: la foto y su descripción deben quedar cerca. Con eso, clasificar sin entrenar.

CLIP: texto e imagen en el MISMO espacio de vectores

dos encoders, un contrato: la foto y su descripción deben quedar cerca — todo lo demás, lejos

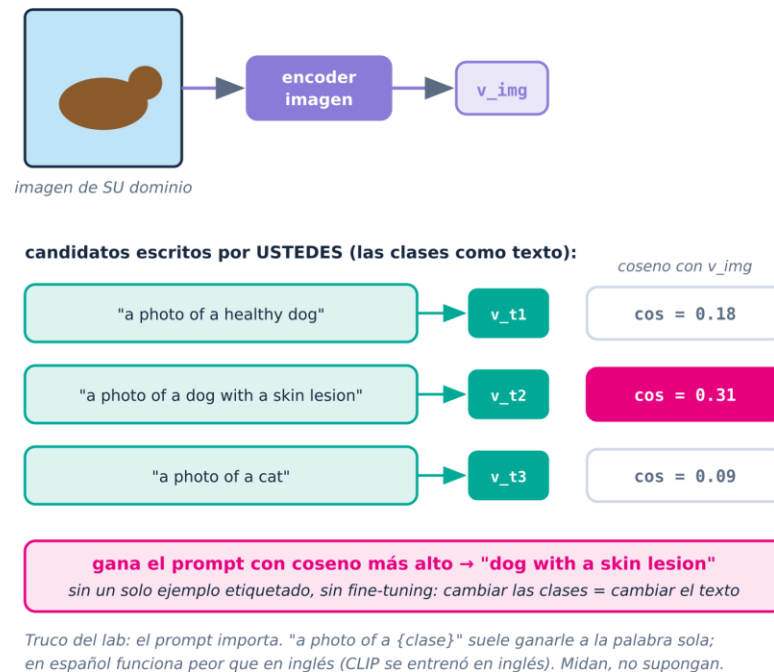
1 · CÓMO SE ENTRENA: contraste sobre 400 M pares (imagen, texto) de internet



Lo que sale: dos encoders que hablan el mismo idioma vectorial

El vector de la foto de un perro y el vector de la frase "a photo of a dog" quedan CERCA.
Es el mismo coseno de S05 (BERTScore), S06 (harness) y S07 (retrieval) — ahora entre modalidades.
CLIP no clasifica: mide parecido. La clasificación la construimos nosotros con prompts →

2 · ZERO-SHOT: clasificar sin entrenar nada



Qué pueden hacer con CLIP mañana en su proyecto:

clasificar

etiquetas como texto: "recibo", "producto", "lesión"

buscar imágenes con texto

el retrieval de S07 pero sobre fotos (Vision RAG)

filtrar / moderar

¿esta foto es de un animal o de una persona?

dar contexto al RAG

el resultado de CLIP entra al prompt como texto

Radford et al., 2021 · "Learning Transferable Visual Models from Natural Language Supervision"

el coseno que ya conocen, entre una foto y una frase — cuarta vez que la misma herramienta cambia de oficio

Las clases son texto: escribanlas.

EQUIPO / DOMINIO	IMAGEN DE ENTRADA	PROMPTS CANDIDATOS (en inglés rinde mejor)
Educación	foto de un ejercicio / cuaderno	"a handwritten math exercise" · "a printed textbook page" · "a diagram"
E-commerce	foto de producto	"a photo of a blender" · "a photo of shoes" · "a product with visible damage"
Salud pública	foto de campo o documento	"a mosquito breeding site" · "a vaccination card" · "a skin rash"
Agro / clima	foto de cultivo	"a coffee leaf with rust" · "a healthy coffee leaf" · "dry cracked soil"
Legal	documento escaneado	"a scanned contract" · "an ID document" · "a handwritten note"
Vet / turismo	foto de mascota o documento	"a dog" · "a cat" · "a pet passport" · "a dog with a skin lesion"

El prompt es un hiperparámetro: "a photo of a X" vs. "X" vs. "a close-up photo of a X, veterinary" cambian el resultado. Y se mide igual que todo: con ejemplos etiquetados por ustedes (¿les suena? es un eval set).



Al notebook • Lab A

CLIP zero-shot sobre imágenes de su dominio

Cargan CLIP, vectorizan 3–6 imágenes (las de ejemplo o las suyas) y un conjunto de prompts-clase. Calculan el coseno imagen–texto, ven la matriz completa y prueban dos formulaciones de prompt para la misma clase. Cierran con una mini-búsqueda: una frase encuentra la imagen (Vision RAG en 5 líneas).

```
cargar CLIP → embed(imágenes) • embed(prompts) → coseno → argmax → cambiar el prompt → medir •  
S11_Lab_Vision.ipynb
```

Clase grabada: pausen el video aquí, trabajen el Lab A con su equipo (~20 min) y vuelvan.

Qué vieron con CLIP.

- **La diagonal apareció sola** — los pares (imagen, prompt) correctos tuvieron el coseno más alto sin entrenar nada: eso es lo que compró el contraste sobre 400 M pares.
- **El prompt movió la aguja** — "a photo of a dog" y "dog" no dan lo mismo; el contexto del prompt es parte del modelo, igual que en S07.
- **Buscar imágenes con texto es el retrieval de S07** — mismo coseno, misma base vectorial; solo cambió qué se vectoriza. Un chunk puede ser una foto.
- **Y falló donde esperábamos** — clases parecidas entre sí o vocabulario muy de dominio: ahí CLIP necesita fine-tuning (S04) o un modelo que lea (siguiente bloque).

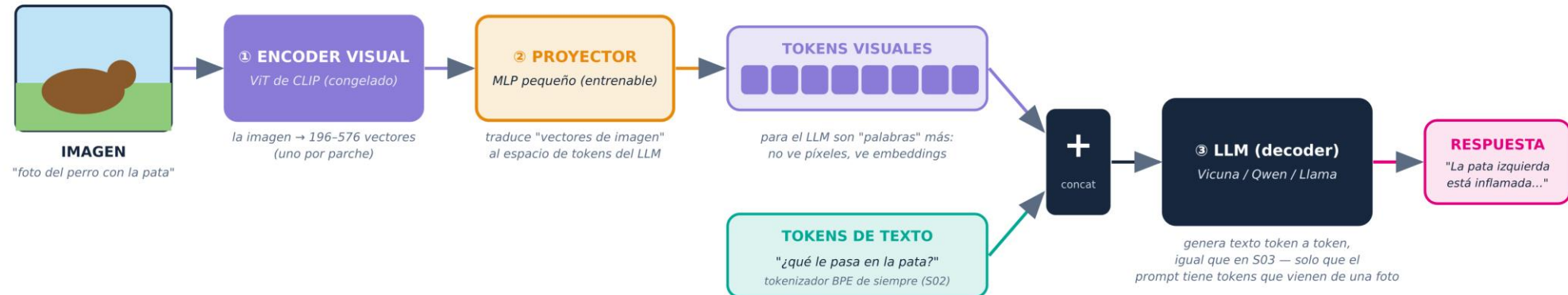
Anoten la clase donde CLIP se equivocó: es su primer caso de prueba visual para el eval set de M4.

Un LLM que mira.

CLIP mide parecido. Para describir, responder y razonar sobre una imagen hace falta el generador — y una forma de meterle la foto al prompt.

Un LLM que mira: la receta de los modelos multimodales

LLaVA, Qwen-VL, BLIP-2, GPT-4V... distintos nombres, la misma idea: convertir la imagen en tokens que el LLM ya sabe leer



Nada de esto es nuevo para ustedes:

- 1 el encoder visual es el ViT de hoy, entrenado con el contraste de CLIP (bloques 2 y 3).
- 2 el proyector es un MLP (un perceptrón, S03) que cambia de base: de "espacio CLIP" a "espacio del LLM".
- 3 el LLM es el decoder de S03 con el prompt aumentado de S07; solo que parte del contexto es la foto.
- 4 es RAG con otra fuente: en S07 el contexto eran chunks; aquí, tokens visuales. El generador no cambia.

Lo que hacen bien

- describir una foto (captioning)
- responder preguntas sobre ella (VQA)
- leer texto en la imagen (OCR "informal")
- comparar dos imágenes en un mismo prompt
- seguir instrucciones: "responde en JSON"

Donde fallan (y hay que medir)

- contar objetos y leer números pequeños
- detalles finos: alucinan lo que "debería" estar
- posición espacial precisa (izquierda/derecha)
- dominio experto sin contexto (radiografías)
- prompts en español rinden algo peor

Las familias, con la misma receta:

BLIP-2 (2023)

Q-Former como proyector; encoder y LLM congelados

LLaVA (2023)

CLIP-ViT + MLP + Vicuna; "visual instruction tuning"

Qwen2-VL (2024)

resolución variable; 2B corre en Colab: el del lab

GPT-4V / Gemini

cerrados, misma idea a otra escala

Lo que se entrena en LLaVA: primero SOLO el proyector (alinear), luego proyector + LLM con instrucciones visuales. El encoder CLIP casi nunca se toca.

La imagen se vuelve tokens; el LLM hace lo de siempre. Por eso un VLM alucina igual que un LLM: hay que medirlo con su harness.

Liu et al., 2023 · "Visual Instruction Tuning" (LLaVA) · Li et al., 2023 · BLIP-2

Todo lo de prompts aplica; dos cosas nuevas.

Describir antes de preguntar

Pedir primero "describe lo que ves" y después la pregunta reduce alucinaciones: el modelo se ancla en lo que escribió. Es el "pensar antes de actuar" de ReAct, en versión visual.

La válvula de escape también existe aquí

"Responde SOLO con lo que se ve en la imagen. Si no se ve, di: no se aprecia en la imagen." Sin esto, un VLM inventa el número o de la placa, la fecha del documento o la raza del perro.

Salida estructurada

Para su pipeline: "responde en JSON con campos {objeto, estado, confianza}". Lo que sale del VLM entra al RAG o a la tool como texto: es un chunk más.

Lo que un VLM pequeño no hace bien

Contar, leer números pequeños, ubicar izquierda/derecha, diagnosticar sin contexto. Si su caso depende de eso, el patrón no es "pregúntele al VLM" sino "VLM + herramienta especializada" (S13).

Un VLM alucina exactamente como un LLM, porque ES un LLM. Faithfulness aplica: ¿lo que dijo se sostiene en la imagen? El juez, eso sí, tiene que poder ver.



Al notebook • Lab B

Visual QA con un modelo multimodal pequeño

Cargan Qwen2-VL-2B (la receta de LLaVA en tamaño Colab; LLaVA-1.5-7B en 4 bits queda como celda opcional). Le pasan una imagen del dominio y tres tipos de prompt: describir, preguntar con válvula de escape, y extraer JSON. Luego una pregunta trampa sobre algo que NO está en la imagen, y comparan cómo responde con y sin válvula.

```
cargar VLM → imagen + prompt → describir • preguntar • JSON → pregunta trampa → ¿alucinó? •  
S11_Lab_Vision.ipynb
```

Clase grabada: pausen el video, trabajen el Lab B (~25 min) y vuelvan para el cierre.

Qué vieron con el modelo multimodal.

- **Describió bien lo general, dudó en lo fino** — objetos, escena y texto grande, sí; números pequeños y conteos, a veces no. Su eval set visual tiene que incluir ambos.
- **La válvula de escape funcionó (a veces)** — con la instrucción, la pregunta trampa recibió "no se aprecia"; sin ella, el modelo inventó con total seguridad. Igual que en S07.
- **El JSON salió parseable casi siempre** — y "casi" es la palabra: validen como en el lab A de S10. Lo que sale del VLM es texto que entra a su sistema.
- **La latencia no es gratis** — un VLM de 2B tarda más que su generador de texto, y uno de 7B más aún. Se anota, se reporta, se decide.

Ya tienen los dos ladrillos: CLIP (medir parecido) y VLM (leer y describir). Lo que sigue es decidir dónde va cada uno en SU sistema.

¿Y esto, para SU sistema?

Tres patrones para meterle ojos a lo que ya construyeron — y la misma regla de siempre para saber si valió la pena.

Tres formas de conectar la visión: input, output, tool.

VISIÓN COMO INPUT

El usuario manda una foto

La imagen entra al sistema y un VLM la convierte en texto (descripción, JSON) que alimenta al RAG o al agente.

Ej.: vet — foto de la lesión → descripción → RAG sobre protocolos.

VISIÓN COMO TOOL

El agente decide mirar

CLIP o el VLM son una herramienta más (S10): el agente la llama cuando la pregunta lo pide.

Ej.: e-commerce — "¿este producto está dañado?" → tool clasificar(imagen).

VISIÓN COMO OUTPUT

El sistema produce una imagen

Generación con difusión (S12): ilustrar, mostrar, sintetizar.

Ej.: educación — generar la figura del problema de geometría.

Para M4 basta UNO, bien justificado: ¿qué pregunta de SU usuario no se podía responder sin ver? Si la respuesta es "ninguna", el patrón correcto es el más barato y se dice con esa honestidad.

Ideas por dominio — para discutir, no para copiar.

EQUIPO	LA PREGUNTA QUE NECESITA OJOS	PATRÓN	CÓMO SE MIDE
Educación	el estudiante manda la foto de su ejercicio a mano	input · VLM → texto → RAG	¿transcribió bien? acierto vs. gold
E-commerce	"¿qué producto es este?" / "¿llegó dañado?"	tool · CLIP clasifica	accuracy sobre 20 fotos etiquetadas
Salud pública	foto de un criadero / de un carnet de vacunas	input · VLM extrae JSON	campos correctos / total
Agro / clima	foto de la hoja: ¿roya, minador, sana?	tool · CLIP o ViT fine-tuneado	accuracy + casos difíciles
Legal	documento escaneado → ¿qué tipo es? → RAG	input · CLIP enruta, VLM lee	¿enrutó bien? ¿leyó los campos?
Vet / turismo	foto de la mascota o del pasaporte de mascota	input · VLM describe → RAG	faithfulness visual + acierto

Fíjense en la última columna: ninguna dice "se ve bien". Cada componente visual llega con su forma de medirlo — y esa medición entra al harness de M2.

Cómo se mide un componente visual.

El eval set crece: casos con imagen

A sus 10+ gold se suman ≥ 5 casos donde el input incluye una imagen (y la respuesta esperada). Las mismas tres columnas: input, esperado, criterio.

El harness es el mismo

`harness(eval_set, sistema)`: si el sistema recibe imagen, la función no cambia. Las tres dimensiones siguen: similitud, juez, acierto de dominio.

Lo que sí cambia: el juez tiene que ver, o le dan la descripción

Faithfulness visual = ¿lo que dijo se sostiene en la imagen? O el juez es un VLM, o le pasan una descripción gold de la imagen como "contexto" (RAGAS con otro contexto, S10).

Y la latencia entra al scorecard

Un VLM en el camino suma segundos. La pregunta de S08 sigue: ¿cuánto sube el harness por segundo de latencia?

Delta contra el baseline de M3: ¿los casos con imagen ahora se responden? ¿los casos sin imagen no empeoraron? Las dos preguntas, en la misma tabla.

Antes de la próxima sesión: su plan de integración visual.

1**LA PREGUNTA**

¿Qué pregunta de su usuario necesita ver una imagen? Una frase. Si no hay ninguna, díganlo y justifiquen el patrón más barato.

2**EL PATRÓN Y EL MODELO**

input / tool / output · CLIP, ViT fine-tuneado, VLM (Qwen2-VL, LLaVA) o difusión (S12).

3**CÓMO SE MIDE**

≥5 casos con imagen para el eval set y qué dimensión del harness los califica.

Entrega M4 · S13 (15 OCT) · cierra dom 25 OCT · 10%: pipeline multimodal + casos de prueba. El plan de esta semana es el borrador de esa entrega. Y no olviden: M3 cierra ESTE domingo 27 SEP.

Tres nombres más de la misma familia.

SAM • Segment Anything

Un modelo que recorta cualquier objeto con un clic o una caja. Útil como tool: "dame solo la hoja" antes de clasificarla.

DINOv2

Un ViT entrenado sin texto (auto-supervisado) cuyos embeddings sirven para casi todo: buscar imágenes parecidas, agrupar, detectar duplicados. CLIP sin el texto.

ColPali • Vision RAG

Indexar PDFs como IMÁGENES de página (sin extraer texto) y recuperar por parecido visual. Para corpus con tablas, diagramas y escaneos, a veces le gana al chunking de S07.

Fuera del alcance de la entrega. Pero si su corpus legal o de salud está lleno de escaneos, ColPali es una conversación que vale la pena tener en S13.

La próxima sesión: generar imágenes.

Hoy el transformer aprendió a VER. En S12 aprende a DIBUJAR: modelos de difusión (DDPM, latent diffusion, Stable Diffusion) y cómo controlarlos (ControlNet, IP-Adapter). Es el patrón "visión como output".

Traigan

su plan de integración visual (slide anterior) y, si eligen output, 2–3 ejemplos de la imagen que querrían generar.

Recuerden

M3 cierra el domingo 27 SEP a las 23:59: sistema RAG + reporte con scorecard propio y métricas RAGAS.

Semana de receso 5–9 OCT: no hay clase. S12 es el 1 OCT y S13 (entrega M4) el 15 OCT.

Parches en vez de palabras. El resto ya lo sabían.

ViT lee imágenes con el encoder de S03; CLIP pone texto e imagen en el mismo espacio y clasifica sin entrenar; un VLM es un LLM con tokens que vienen de una foto — y alucina como tal. La próxima semana: difusión. Y este domingo: M3.